Wallet Service

Resilience & Integrity Test Report

Report date	24 Oct 2026, 16:33
Branch / commit	master @ b8ff336 (with uncommitted test changes)
Stack under test	Spring Boot 4.1.2, Java 21, PostgreSQL 16, Redis 7, Apache Kafka 3.8 (KRaft)
Test framework	JUnit 5 + AssertJ + Mockito, run with Maven Surefire against the real Docker services
Result	64 passed, 0 failed, 0 skipped out of 64 tests (total suite time 251 s)

ALL TESTS PASSED

1. Executive summary

Every test case from the test plan was turned into an automated integration test that boots the full application against the real PostgreSQL, Redis and Kafka containers. Where the plan calls for an external load tool (k6/JMeter), the equivalent load is generated in-process with a latch that releases all requests at the same instant, which is harsher on locking than a ramped load. Measured values (latencies, counts, backoff gaps) are recorded by the tests themselves and shown next to each case.

Category	Cases	Passed	Failed	Time (s)
1. Concurrency & race conditions (double-spend suite)	7	7	0	16.6
2. Idempotency edge cases	9	9	0	10.9
3. Distributed resilience & event-driven (Kafka, webhooks)	9	9	0	67.0
4. Financial integrity & consistency	8	8	0	31.9
5. Security & load limits	16	16	0	11.9
6. Part 2 - Consumer idempotency & replay (Kafka failure modes)	2	2	0	10.9
7. Part 2 - Isolation level, serialization & deadlock recovery	5	5	0	21.9
8. Part 2 - Financial boundary & precision	5	5	0	14.0

2. Findings and recommendations

All assertions pass. Findings marked FIXED were defects exposed by failing tests in the first run (5 in total); the application was changed and the same tests now pass. The remaining items are open recommendations.

#	Severity	Finding	Recommendation
1	**High** **FIXED**	**Dead letters go to an unprovisioned '-dlt' topic (TC-3.2c failed)** KafkaConfig creates transfer-completed.DLT with 3 partitions and its comment says failed records go there, but DeadLetterPublishingRecoverer in Spring Kafka 4 publishes to <topic>-dlt. The broker auto-creates transfer-completed-dlt with 1 partition; the recoverer warns that the partition does not exist and lets the producer pick one. Anything that monitors or replays .DLT never sees a failed webhook; with auto-topic-creation disabled (common in production) the records would be lost outright.	**Applied:** KafkaConfig builds DeadLetterPublishingRecoverer with an explicit resolver (rec.topic() + '.DLT', same partition). The stray transfer-completed-dlt topic on the dev broker can be deleted.
2	**High** **FIXED**	**Redelivered events send duplicate webhooks (P2-TC-1.1a failed)** WebhookDeliveryConsumer keeps no record of delivered events. A worker that crashes after posting but before committing its offset, a consumer rebalance, or any retry re-sends the webhook: the test merchant received the same event twice. Notifications are safe (P2-TC-1.1b).	**Applied:** V7 migration adds webhook_deliveries (transaction_id, endpoint_id) PK; WebhookDeliveryConsumer skips pairs already delivered and records each success. Webhooks now carry X-Wallet-Event-Id so merchants can dedupe the remaining crash window (between POST and insert).
3	**High** **FIXED**	**Deadlocks and serialization failures are not retried (P2-TC-2.1b failed)** A real 40P01 raised during TransferService.transfer propagates as CannotAcquireLockException; there is no retry. The sorted lock order prevents deadlocks between transfers, but any other writer to wallets (admin scripts, future features, lock timeouts) will fail user requests.	**Applied:** TransferController.transfer is @Retryable(includes = ConcurrencyFailureException, maxRetries = 3, delay = 50 ms, multiplier = 2, jitter = 20 ms), outside the transaction, enabled by ResilienceConfig (@EnableResilientMethods, Spring 7). Exhausted retries return 503 with Retry-After (P2-TC-2.1e).

#	Severity	Finding	Recommendation
4	Medium FIXED	**Unhandled server errors reach the client as 401 with an empty body** Seen in P2-TC-2.1b: when a controller throws, Tomcat forwards to /error, which falls under anyRequest().authenticated(). The stateless error dispatch has no authentication, so a logged-in user gets 401. Clients will treat a database fault as an expired session and log out.	**Applied:** /error is permitAll() in SecurityConfig; GlobalExceptionHandler maps ConcurrencyFailureException to 503 problem+json. Regression test P2-TC-2.1c.
5	High FIXED	**Fractional amounts are silently truncated (P2-TC-3.1c failed)** amount is a long in paise, and Jackson coerces JSON floats to integers. {"amount": 33.33} transfers 0.33 rupees (33 paise) and returns 201; 3333.999 becomes 3333. A client that sends rupees as a decimal moves the wrong amount with no error.	**Applied:** application.yml sets spring.jackson.deserialization.accept-float-as-int: false; every fractional amount now gets 400.
6	High	**Events produced during a Kafka broker outage are lost** TransferEventPublisher sends after commit and only logs a failure. With the broker unreachable (TC-3.1e) the transfer commits but its event is dropped, so notifications and webhooks never happen. Test-plan 3.1 ('events aren't lost') holds for worker outages (TC-3.1a) but not for broker outages at publish time.	Write the event to an outbox table in the same DB transaction as the transfer and publish it from a relay (polling or Debezium), marking rows sent on ack.
7	High	**Broker outage stalls every transfer response and holds a DB connection** afterCommit runs on the request thread before the JDBC connection is released, and KafkaProducer.send blocks for max.block.ms waiting for metadata (default 60 s). TC-3.1e/f measured this with max.block.ms=3 s. With defaults, each transfer during an outage takes ~60 s and pins one of the 10 Hikari connections, so the whole API (reads too) degrades within seconds.	The outbox in finding 6 removes Kafka from the request path. As a stop-gap set spring.kafka.producer.properties.max.block.ms to ~1000 and hand the send to an executor.
8	Low	**Key reuse with a different payload returns 422, not 400** TC-2.2a: the plan expects 400; the API returns 422 Unprocessable Content. 422 is what the IETF 'Idempotency-Key HTTP Header' draft specifies for this case, so the behaviour is defensible.	Keep 422 and update the test plan / API documentation.
9	Medium	**Rate limiter fails open when Redis is unavailable** TC-5.2e: any Redis error allows the request. During a Redis outage, /auth/login loses its brute-force protection entirely.	Fail closed for /auth/login (or fall back to an in-memory per-instance bucket such as Bucket4j).
10	Low	**Login limit keys on the socket address** Code review: RateLimitFilter uses request.getRemoteAddr(). Behind a load balancer or proxy every client shares one address, so 10 failed logins anywhere lock everyone out for a minute.	Enable server.forward-headers-strategy=native (or framework) behind a trusted proxy.
11	Low	**Dead-letter topic has no consumer** Now that dead letters reach .DLT (finding 1), still nothing reads or replays the dead-letter topic. The per-endpoint delivery log added for finding 2 also stops retries from re-sending to endpoints that already succeeded.	Add an admin endpoint or job that replays .DLT records once the merchant endpoint is healthy, and alert on DLT growth.
12	Info	**Access tokens are trusted until expiry** Code review: JwtAuthFilter does not consult the database, so a disabled user or demoted admin keeps access for up to 15 minutes. Not exercised by a test.	Acceptable for a 15-minute TTL; if instant revocation is needed, add a token version on the user.
13	Info	**Soak duration scaled down** TC-4.1 runs for 20 s by default instead of the 1 hour in the plan, to keep the suite fast. The invariants checked are identical.	Run -Dchaos.seconds=3600 -Dtest=LedgerIntegrityTest nightly in CI.

3. Test case details

3.1 Concurrency & race conditions (double-spend suite)

TC-1.1a	Concurrent drain of a 100.00 balance
Test	`DoubleSpendSuiteTest#concurrentDrain_exactlyAffordableTransfersSucceed`
Goal	Row locking (SELECT ... FOR UPDATE) must stop a balance being spent twice.
How it runs	User A holds 100.00 (10,000 paise). 50 POST /api/v1/transfers requests of 10.00 each are released at the same instant over HTTP.
Expected	Exactly 10 x 201 and 40 x 422 'Insufficient funds'. A ends at 0, B at 100.00, no 5xx, wallet balances equal their ledger sums, ledger nets to zero.
Actual	Measured: requests = **50** succeeded_201 = **10** rejected_422 = **40** elapsed_ms = **990**
Result	**PASSED** 1.60 s

TC-1.1b	Bidirectional A<->B transfers (deadlock test)
Test	`DoubleSpendSuiteTest#opposingDirections_noDeadlockNoNegativeBalance`
Goal	Locking both wallets in a fixed (sorted id) order must prevent deadlocks.
How it runs	A starts with 100.00, B with 0. 50 A->B and 50 B->A transfers of 10.00 are fired simultaneously (100 requests). B can only spend what it has received at that moment.
Expected	Every reply is 201 or 422 (a deadlock would surface as 500). No negative balance. A + B = 100.00 and each balance equals its ledger sum.
Actual	Measured: requests = **100** a_to_b_succeeded = **50** b_to_a_succeeded = **42** rejected_422 = **8** server_errors_5xx = **0** elapsed_ms = **4536**
Result	**PASSED** 5.47 s

TC-1.1c	Three-wallet ring A->B->C->A
Test	`DoubleSpendSuiteTest#ringOfThreeWallets_noDeadlock`
Goal	Lock ordering must also hold for cycles longer than two wallets.
How it runs	Three wallets with 50.00 each; 60 simultaneous transfers of 1.00 around the ring.
Expected	All 60 return 201, every wallet ends at 50.00, ledger balanced.
Actual	Measured: requests = **60** succeeded_201 = **60**
Result	**PASSED** 4.33 s

TC-1.2	Rapid-fire idempotency flooding
Test	`DoubleSpendSuiteTest#idempotencyFlood_executesOnceAndReplaysSamePayload`
Goal	A retry storm with one Idempotency-Key must execute exactly one transfer.
How it runs	100 identical POST /transfers requests with the same Idempotency-Key released at once; then a different transfer, then a late retry of the original key.
Expected	All 100 return 201 with a byte-identical body; one transfer, one idempotency row, one event published. The late retry returns the original payload even though the balance has moved.
Actual	Measured: requests = **100** distinct_payloads = **1** transfers_executed = **1** elapsed_ms = **508**
Result	**PASSED** 1.67 s

TC-1.R1	Service-level overdraw race (regression)
Test	`TransferConcurrencyTest#concurrentTransfers_neverOverdraw`
Goal	Same invariant as TC-1.1a at the service layer.
How it runs	30 parallel TransferService calls of 10.00 against 100.00.
Expected	10 succeed, 20 rejected with 422, sender at 0.
Actual	As expected.
Result	**PASSED** 0.95 s

TC-1.R2	Service-level opposing transfers (regression)
Test	`TransferConcurrencyTest#opposingTransfers_doNotDeadlock`
Goal	No deadlock for opposing transfers at the service layer.
How it runs	20 parallel transfers, half A->B and half B->A.
Expected	All 20 succeed; balances unchanged.
Actual	As expected.
Result	**PASSED** 1.73 s

TC-1.R3	Service-level key flood (regression)
Test	`TransferConcurrencyTest#sameIdempotencyKey_movesMoneyOnce`
Goal	Same invariant as TC-1.2 at the service layer.
How it runs	10 parallel calls with one key.
Expected	One transaction id, money moved once.
Actual	As expected.
Result	**PASSED** 0.81 s

3.2 Idempotency edge cases

TC-2.1a	Interrupted request: response lost after commit
Test	`IdempotencyEdgeCaseTest#clientTimeoutAfterCommit_retryReturnsOriginalResult`
Goal	A client that never saw the response must be able to retry safely.
How it runs	A separate DB connection locks the recipient wallet so the server stalls; the client gives up after 1 s (HttpTimeoutException). The lock is released, the abandoned request commits on the server, then the client retries with the same key.
Expected	Retry returns 201 with the transaction id that was committed while the client was gone; exactly one transfer; balances 90.00 / 10.00.
Actual	As expected.
Result	**PASSED** 2.25 s

TC-2.1b	Retry while the original is still in flight
Test	`IdempotencyEdgeCaseTest#retryWhileOriginalInFlight_getsSameResult`
Goal	A retry that overlaps the original must not execute a second transfer.
How it runs	Original and retry are both sent while the recipient wallet is locked; both are verified to be pending, then the lock is released.
Expected	Both return 201 with identical bodies; one transfer.
Actual	As expected.
Result	**PASSED** 1.96 s

TC-2.1c	Failure before commit does not burn the key
Test	`IdempotencyEdgeCaseTest#failedRequestRollsBackKey_retryExecutes`
Goal	A crash/abort mid-transaction must roll back the key claim together with the money movement.
How it runs	Transfer with insufficient funds (key is claimed, then the transaction aborts). Top up, retry with the same key.
Expected	First call 422 and no key row persisted; retry executes (201) exactly once.
Actual	As expected.
Result	**PASSED** 0.95 s

TC-2.2a	Payload mismatch with existing key (50.00 then 500.00)
Test	`IdempotencyEdgeCaseTest#sameKeyDifferentAmount_isRejected`
Goal	Reusing a key with different parameters must be rejected.
How it runs	Idempotency-Key abc-123-... transfers 50.00, then the same key attempts 500.00.
Expected	Test plan: 400. Implemented: 422 Unprocessable Content with detail 'Idempotency-Key was already used with a different request' (see finding 8). Only 50.00 moves.
Actual	Measured: second_request_status = **422**
Result	**PASSED** 0.99 s

TC-2.2b	Same key, different recipient
Test	`IdempotencyEdgeCaseTest#sameKeyDifferentRecipient_isRejected`
Goal	The request fingerprint must include the recipient.
How it runs	Same key and amount, second request targets a different user.
Expected	Second request 422; second recipient receives nothing.
Actual	As expected.
Result	**PASSED** 1.23 s

TC-2.2c	Recipient e-mail case is normalised
Test	`IdempotencyEdgeCaseTest#normalisedEmail_isTreatedAsSameRequest`
Goal	Cosmetic differences must not be treated as a different request.
How it runs	Same key, recipient e-mail upper-cased on the retry.
Expected	Retry is a replay (201, identical body); money moves once.
Actual	As expected.
Result	**PASSED** 0.73 s

TC-2.2d	Keys are scoped per user
Test	`IdempotencyEdgeCaseTest#keysAreScopedPerUser`
Goal	Two users choosing the same key must not collide.
How it runs	Users A and C both send the same Idempotency-Key value.
Expected	Both execute with different transaction ids.
Actual	As expected.
Result	**PASSED** 1.49 s

TC-2.2e	Missing / blank / over-long key header
Test	`IdempotencyEdgeCaseTest#invalidKeyHeader_isRejected`
Goal	Transfers without a usable key must be refused.
How it runs	No header, a single space, and a 101-character key.
Expected	400 for each; balance untouched.
Actual	As expected.
Result	**PASSED** 0.76 s

TC-2.R1	Key reuse with different body (regression)
Test	`TransferConcurrencyTest#reusedKeyWithDifferentBody_isRejected`
Goal	Service-level version of TC-2.2a.
How it runs	Two calls, same key, amounts 1.00 and 2.00.
Expected	Second call 422.
Actual	As expected.
Result	**PASSED** 0.53 s

3.3 Distributed resilience & event-driven (Kafka, webhooks)

TC-3.1a	Worker outage: transfers succeed, backlog processed on restart
Test	`EventPipelineTest#workersDown_transfersSucceed_eventsProcessedAfterRestart`
Goal	The payment path must not depend on the consumers, and no event may be lost while they are down.
How it runs	The application's listener containers are stopped. Two private worker groups commit their offset at the current end of the topic (a worker that crashed just before the transfers). 5 transfers are made to a merchant with a registered webhook; a probe consumer verifies the 5 events are in the topic. The real NotificationConsumer and WebhookDeliveryConsumer beans are then restarted on those groups with the application's error handler. (Private groups keep the locally running app, which joins the real groups, from stealing partitions.)
Expected	All transfers commit while workers are down; nothing is processed until restart; after restart 10 notifications (sender + receiver) are stored and 5 signed webhooks delivered.
Actual	Measured: transfers_while_down = **5** events_found_in_kafka = **5** notifications_after_restart = **10** webhooks_after_restart = **5** backlog_drain_ms = **599**
Result	**PASSED** 3.91 s

TC-3.1b	At-least-once redelivery does not duplicate notifications
Test	`EventPipelineTest#redeliveredEvent_isIdempotent`
Goal	Kafka may redeliver; consumers must be idempotent.
How it runs	The same TransferCompletedEvent is fed to the notification consumer three times.
Expected	Exactly 2 notification rows (one per party).
Actual	As expected.
Result	**PASSED** 0.77 s

TC-3.1c	Events are published only after commit
Test	`EventPipelineTest#publisher_sendsOnlyAfterCommit`
Goal	Consumers must never see an event for a rolled-back transfer.
How it runs	TransferEventPublisher with a mocked KafkaTemplate inside a transaction that rolls back, then one that commits.
Expected	No send on rollback; exactly one send on commit.
Actual	As expected.
Result	**PASSED** 0.43 s

TC-3.1d	Publish failure never fails the transfer
Test	`EventPipelineTest#publisherFailure_isSwallowed`
Goal	A broker error after commit must not surface as an API error.
How it runs	KafkaTemplate.send throws inside afterCommit.
Expected	No exception escapes the transaction.
Actual	As expected.
Result	**PASSED** 0.03 s

TC-3.1e	Kafka broker down: POST /transfers still returns 201
Test	`KafkaOutageTest#brokerDown_transferStillSucceeds`
Goal	Decoupled architecture: a broker outage must not block payments.
How it runs	Application booted with bootstrap-servers pointing at a closed port and max.block.ms=3000. A transfer and an idempotent retry are sent over HTTP.
Expected	201, balances and ledger correct, retry replays. Latency is bounded by max.block.ms (see findings 6 and 7: the event itself is dropped).
Actual	Measured: max_block_ms = **3000** transfer_latency_during_outage_ms = **3238**
Result	**PASSED** 4.29 s

TC-3.1f	Kafka broker down: concurrent transfers
Test	`KafkaOutageTest#brokerDown_concurrentTransfersAllCommit`
Goal	Measure the effect of the producer's metadata wait under concurrency.
How it runs	20 users each send one transfer simultaneously while the broker is unreachable.
Expected	All 20 commit with 201. Elapsed time shows the request threads (and DB connections) held for max.block.ms each (see finding 7).
Actual	Measured: concurrent_transfers = **20** elapsed_ms = **6649** max_block_ms = **3000**
Result	**PASSED** 17.25 s

TC-3.2a	Merchant webhook fails 3x (HTTP 500), succeeds on attempt 4
Test	`WebhookRetryTest#failsThreeTimesThenSucceeds_withExponentialBackoff`
Goal	Unstable merchant servers must not lose webhooks; retries must back off exponentially.
How it runs	Local merchant endpoint scripted to answer 500, 500, 500, 200. The real WebhookDeliveryConsumer runs in a listener container with the application's DefaultErrorHandler on a private topic.
Expected	Exactly 4 attempts with gaps of ~1 s, ~2 s, ~4 s; each attempt carries the same body and a valid HMAC-SHA256 signature; nothing in the DLT; no 5th attempt.
Actual	Measured: attempts = **4** gap1_ms = **1035** gap2_ms = **2045** gap3_ms = **4027** successful_deliveries = **1** dlt_records = **0**
Result	**PASSED** 14.44 s

TC-3.2b	Merchant webhook times out once, then succeeds
Test	`WebhookRetryTest#timeoutThenSuccess`
Goal	A hanging endpoint must be treated as a failure and retried.
How it runs	First attempt hangs for 7 s (client timeout is 5 s), second answers 200.
Expected	2 attempts, delivered, DLT empty.
Actual	Measured: attempts = **2** gap1_ms = **6035**
Result	**PASSED** 13.31 s

TC-3.2c	Merchant never recovers: event parked in DLT
Test	`WebhookRetryTest#permanentFailure_goesToDeadLetterTopic`
Goal	Retries must be bounded and failed events preserved.
How it runs	Endpoint always answers 500.
Expected	4 attempts, then exactly one record on <topic>.DLT (the dead-letter topic KafkaConfig provisions) carrying the original payload and the exception message.
Actual	FIXED (finding 1). Before the fix the record went to <topic>-dlt, the recoverer's default name, which the broker auto-created with 1 partition. KafkaConfig now passes an explicit destination resolver; the record lands on <topic>.DLT. Measured: attempts = **4** records_in_provisioned_.DLT = **1** records_in_unprovisioned_-dlt = **0**
Result	**PASSED** 12.54 s

3.4 Financial integrity & consistency

TC-4.1	Invariant balance check after a chaotic soak
Test	`LedgerIntegrityTest#chaosThenReconciliation_reportsNoDiscrepancies`
Goal	Prove the double-entry ledger never drifts from wallet balances.
How it runs	16 workers x 20 users run a random mix for the soak duration: top-ups, transfers (many overdrawing), self-transfers, unknown recipients, idempotent replays and key reuse with a different body. The reconciliation job is then triggered through POST /api/v1/admin/reconciliation/runs with an ADMIN token.
Expected	No unexpected errors; every balance equals an independently computed model and its SUM(credits) - SUM(debits); no running balance ever negative; all wallets sum to zero; reconciliation status OK with 0 issues.
Actual	Measured: duration_s = **20** workers = **16** users = **20** operations = **3206** ops_per_second = **160** outcome.key_reuse_different_body:422 = **257** outcome.replay:ok = **306** outcome.self_transfer:400 = **74** outcome.skipped:ok = **7** outcome.topup:ok = **632** outcome.transfer:422 = **37** outcome.transfer:ok = **1653** outcome.unknown_recipient:404 = **240** reconciliation_status = **OK** reconciliation_issues = **0** wallets_checked = **684** transactions_checked = **11577**
Result	**PASSED** 27.11 s

TC-4.2	Reconciliation detects corruption
Test	`LedgerIntegrityTest#reconciliation_detectsInjectedCorruption`
Goal	The checker must actually catch drift (no false negatives).
How it runs	Insert a COMPLETED transaction without ledger entries and an extra 0.01 credit on a real transaction, run reconciliation, read the issues via the admin API, then clean up.
Expected	Status MISMATCH with TRANSACTION_WITHOUT_LEDGER, UNBALANCED_TRANSACTION, WALLET_BALANCE_MISMATCH and LEDGER_NOT_ZERO_SUM.
Actual	Measured: issues_detected = **4**
Result	**PASSED** 1.78 s

TC-4.3	Only one reconciliation run at a time
Test	`LedgerIntegrityTest#concurrentRun_isRefused`
Goal	The Redis lock must stop overlapping runs across instances.
How it runs	Hold lock:reconciliation in Redis, call the admin API, release, call again.
Expected	409 while the lock is held, 200 afterwards.
Actual	As expected.
Result	**PASSED** 0.46 s

TC-4.4	Invalid amounts never touch the ledger
Test	`LedgerIntegrityTest#invalidAmounts_neverReachTheLedger`
Goal	Validation must stop zero, negative and oversized amounts before any write.
How it runs	Transfers of 0, -5.00 and 100,000.01; top-up of -0.01.
Expected	400 for each; ledger row count and balance unchanged.
Actual	As expected.
Result	**PASSED** 0.74 s

TC-4.R1	Clean ledger reports OK (regression)
Test	`ReconciliationTest#cleanLedger_reportsOk`
Goal	Baseline reconciliation.
How it runs	One transfer, run reconciliation.
Expected	OK, 0 issues.
Actual	As expected.
Result	**PASSED** 0.54 s

TC-4.R2	Corrupted balance detected (regression)
Test	`ReconciliationTest#corruptedBalance_isDetected`
Goal	Detect a wallet balance edited outside the ledger.
How it runs	Add 5.00 to a balance directly.
Expected	MISMATCH with WALLET_BALANCE_MISMATCH.
Actual	As expected.
Result	**PASSED** 0.43 s

TC-4.R3	Insufficient funds leaves balances untouched (regression)
Test	`TransferConcurrencyTest#insufficientFunds_isRejectedAndLeavesBalancesUntouched`
Goal	Rejected transfer writes nothing.
How it runs	Transfer 10.00 from a 5.00 wallet.
Expected	422, balances unchanged.
Actual	As expected.
Result	**PASSED** 0.63 s

TC-4.R4	Self-transfer rejected (regression)
Test	`TransferConcurrencyTest#transferToSelf_isRejected`
Goal	No-op transfers are refused.
How it runs	Transfer to own e-mail.
Expected	400.
Actual	As expected.
Result	**PASSED** 0.22 s

3.5 Security & load limits

TC-5.1a	Expired JWT
Test	`SecurityBoundaryTest#expiredToken_isUnauthorized`
Goal	Expired tokens must be refused.
How it runs	Token correctly signed but expired an hour ago.
Expected	401; victim balance untouched.
Actual	As expected.
Result	**PASSED** 0.58 s

TC-5.1b	Signature-tampered JWT
Test	`SecurityBoundaryTest#tamperedSignature_isUnauthorized`
Goal	Any change to the signature must invalidate the token.
How it runs	One signature character flipped.
Expected	401.
Actual	As expected.
Result	**PASSED** 0.89 s

TC-5.1c	Payload swapped to another user
Test	`SecurityBoundaryTest#payloadSwappedToVictim_isUnauthorized`
Goal	An attacker must not impersonate a victim by editing the subject.
How it runs	Attacker's token with the payload replaced by the victim's id, original signature kept.
Expected	401; victim not debited.
Actual	As expected.
Result	**PASSED** 0.57 s

TC-5.1d	JWT signed with a foreign key
Test	`SecurityBoundaryTest#foreignKey_isUnauthorized`
Goal	Only tokens signed with the server secret are accepted.
How it runs	Token signed with another HMAC key.
Expected	401.
Actual	As expected.
Result	**PASSED** 0.45 s

TC-5.1e	Unsigned 'alg: none' JWT
Test	`SecurityBoundaryTest#algNone_isUnauthorized`
Goal	Classic alg=none bypass must fail.
How it runs	Header {alg: none}, empty signature.
Expected	401.
Actual	As expected.
Result	**PASSED** 0.65 s

TC-5.1f	Malformed, missing or wrong-scheme token
Test	`SecurityBoundaryTest#malformedOrMissing_isUnauthorized`
Goal	Anything other than a valid Bearer token is unauthenticated.
How it runs	'not.a.jwt', no Authorization header, 'Basic <jwt>'.
Expected	401 each.
Actual	As expected.
Result	**PASSED** 0.85 s

TC-5.1g	Role claim escalated to ADMIN
Test	`SecurityBoundaryTest#roleEscalation_isUnauthorized`
Goal	Users cannot grant themselves admin rights.
How it runs	Payload edited to role=ADMIN with the original signature; also the genuine USER token.
Expected	401 for the forged token, 403 for the genuine USER token on the admin API.
Actual	As expected.
Result	**PASSED** 0.48 s

TC-5.1h	User A attempts to debit user B's wallet
Test	`SecurityBoundaryTest#cannotDebitAnotherUsersWallet`
Goal	The debited wallet must come only from the authenticated principal.
How it runs	A's token with body fields fromUserId / fromWalletId / senderId set to B.
Expected	The API has no way to name a source wallet: the spoofed fields are ignored, A is debited, B is untouched. B's token only ever sees B's wallet.
Actual	As expected.
Result	**PASSED** 1.49 s

TC-5.R1	No token -> 401 (regression)
Test	`SecurityHttpTest#noToken_isUnauthorized`
Goal	Baseline.
How it runs	GET /users/me.
Expected	401.
Actual	As expected.
Result	**PASSED** 0.03 s

TC-5.R2	USER role on admin API -> 403 (regression)
Test	`SecurityHttpTest#userRole_isForbiddenOnAdminEndpoints`
Goal	Baseline.
How it runs	POST admin runs with USER token.
Expected	403.
Actual	As expected.
Result	**PASSED** 0.14 s

TC-5.R3	Refresh token rotation (regression)
Test	`SecurityHttpTest#refreshToken_worksOnlyOnce`
Goal	Refresh tokens are single use.
How it runs	Refresh twice with the same token.
Expected	Second refresh rejected.
Actual	As expected.
Result	**PASSED** 0.20 s

TC-5.2a	Login flood: 500 simultaneous requests from one IP
Test	`RateLimitTest#loginBurst_isThrottled`
Goal	Brute force on /auth/login must be throttled.
How it runs	500 POST /api/v1/auth/login requests released at once from 127.0.0.1 (limit 10/min/IP).
Expected	Exactly 10 reach authentication (401), 490 get 429 with Retry-After 1-60 s and a problem+json body; the Redis counter has a TTL.
Actual	Measured: requests = **500** allowed = **10** throttled_429 = **490** elapsed_ms = **733** requests_per_second = **682**
Result	**PASSED** 0.94 s

TC-5.2b	Transfer flood from one user
Test	`RateLimitTest#transferBurst_isThrottledPerUser`
Goal	Per-user transfer limit (15/min) must hold under concurrency without penalising others.
How it runs	50 simultaneous transfers from one user, then one transfer from another user.
Expected	Exactly 15 x 201 and 35 x 429; only 15 transfers move money; the other user gets 201.
Actual	Measured: requests = **50** allowed_201 = **15** throttled_429 = **35**
Result	**PASSED** 1.99 s

TC-5.2c	Registration limit
Test	`RateLimitTest#registerBurst_isThrottled`
Goal	Account-creation spam is throttled.
How it runs	8 registrations from one IP (limit 5/min).
Expected	5 x 201, 3 x 429.
Actual	As expected.
Result	**PASSED** 1.45 s

TC-5.2d	Reads are not throttled
Test	`RateLimitTest#readsAreNotThrottled`
Goal	Limits apply only to write endpoints.
How it runs	40 simultaneous GET /wallets/me.
Expected	All 200.
Actual	As expected.
Result	**PASSED** 0.59 s

TC-5.2e	Limiter behaviour when Redis is down
Test	`RateLimitTest#redisDown_failsOpen`
Goal	Document the failure mode of the limiter.
How it runs	RateLimiter with a Redis client that throws.
Expected	Requests are allowed (fail-open). See finding 9.
Actual	As expected.
Result	**PASSED** 0.63 s

3.6 Part 2 - Consumer idempotency & replay (Kafka failure modes)

P2-TC-1.1a	**Webhook worker crashes after firing, before committing the offset**
Test	`ConsumerReplayTest#webhookWorkerCrash_merchantReceivesEventOnce`
Goal	A redelivered TransferCompleted event must not make the merchant receive the same webhook twice.
How it runs	Real WebhookDeliveryConsumer on a private topic in a MANUAL-ack container: it delivers the webhook and is stopped without acknowledging (the offset state a killed worker leaves; verified: no committed offset). A new worker on the same group receives the redelivered record.
Expected	Merchant receives the webhook exactly once (the service tracks delivered event ids).
Actual	FIXED (finding 2). Before the fix the merchant received the event twice. Successful deliveries are now recorded in webhook_deliveries (V7 migration) per (transaction, endpoint) and skipped on redelivery; each webhook also carries an X-Wallet-Event-Id header. Measured: webhook_deliveries_of_one_event = **1** deliveries_byte_identical = **true**
Result	**PASSED** 3.35 s

P2-TC-1.1b	**Notification worker crashes before committing the offset**
Test	`ConsumerReplayTest#notificationWorkerCrash_noDuplicateNotifications`
Goal	Redelivery must not create duplicate notifications.
How it runs	Same crash simulation with the real NotificationConsumer after a real transfer.
Expected	The event is processed twice, yet exactly 2 notification rows exist (existence check plus UNIQUE(transaction_id, user_id)).
Actual	Measured: times_event_processed = **2** notification_rows = **2**
Result	**PASSED** 7.57 s

3.7 Part 2 - Isolation level, serialization & deadlock recovery

P2-TC-2.1a	**Hot wallet with an artificial delay inside the transaction**
Test	`ContentionRecoveryTest#hotWalletWithInjectedDelay_allSucceed`
Goal	Pessimistic locking must stay correct and error-free under heavy contention on one row.
How it runs	A test-only PostgreSQL trigger adds pg_sleep(20 ms) to every ledger insert, holding row locks about 40 ms per transfer. 20 users pay a hot wallet while it pays each of them back: 40 simultaneous transfers through the API.
Expected	All 40 return 201, no 5xx, balances exact, ledger balanced. Isolation is READ COMMITTED with SELECT ... FOR UPDATE, so 40001 serialization failures cannot occur on this path.
Actual	Measured: concurrent_transfers = **40** injected_delay_per_ledger_insert_ms = **20** succeeded_201 = **40** server_errors_5xx = **0** elapsed_ms = **5473** isolation_level = **read committed**
Result	**PASSED** 17.45 s

P2-TC-2.1b	Real PostgreSQL deadlock (40P01) during a transfer
Test	`ContentionRecoveryTest#forcedDeadlock_isRetriedNot500`
Goal	A deadlock or serialization failure must be retried with backoff, not returned as an error.
How it runs	A second DB connection locks the wallet the app locks second, waits until the app holds its first lock, then requests that one too. PostgreSQL detects the cycle after deadlock_timeout (1 s) and aborts the application's transaction (verified). The external locks are then released, so a retry could succeed.
Expected	201 after an internal retry; money moved once.
Actual	FIXED (findings 3 and 4). Before the fix the 40P01 escaped and the client got an empty 401. TransferController.transfer is now @Retryable on ConcurrencyFailureException (3 retries, 50 ms exponential backoff with jitter), so the retry succeeds once the lock is released. Measured: deadlock_timeout = **1s** deadlock_victim = **application** api_status = **201**
Result	**PASSED** 1.99 s

P2-TC-2.1c	Unhandled server error seen by a logged-in client
Test	`ErrorResponseTest#unhandledError_is500`
Goal	A server fault must be reported as a server fault, never as an authentication failure.
How it runs	TransferService mocked to throw IllegalStateException; POST /transfers with a valid token.
Expected	500 (before the fix: empty 401, because /error was not permitted). Regression test for finding 4.
Actual	Measured: status_for_unhandled_error = **500**
Result	**PASSED** 0.59 s

P2-TC-2.1d	Transient lock conflict is retried
Test	`ErrorResponseTest#transientConflict_isRetried`
Goal	Deadlock/serialization losers are retried transparently.
How it runs	TransferService mocked to throw CannotAcquireLockException twice, then succeed.
Expected	201 after exactly 3 attempts.
Actual	Measured: attempts_until_success = **3**
Result	**PASSED** 0.97 s

P2-TC-2.1e	Conflict that outlasts the retries
Test	`ErrorResponseTest#persistentConflict_is503`
Goal	When retries are exhausted the client gets an actionable, retryable error.
How it runs	TransferService mocked to always throw CannotAcquireLockException.
Expected	4 attempts (1 + 3 retries), then 503 problem+json with Retry-After: 1.
Actual	Measured: attempts = **4** status = **503**
Result	**PASSED** 0.88 s

3.8 Part 2 - Financial boundary & precision

P2-TC-3.1a	33.33 transferred three times
Test	`PrecisionBoundaryTest#thirtyThreeThirtyThreeThreeTimes`
Goal	No fractional paise may be lost or created.
How it runs	99.99 sent as three transfers of 33.33 to three users.
Expected	Sender exactly 0.00, each receiver 33.33, wallets equal ledger sums.
Actual	As expected.
Result	**PASSED** 1.78 s

P2-TC-3.1b	Three-way split and 200 one-paisa transfers
Test	`PrecisionBoundaryTest#splitsAndMicroTransfersStayExact`
Goal	Uneven splits and the smallest unit stay exact under concurrency.
How it runs	100.00 split as 33.33 + 33.33 + 33.34; then 200 simultaneous transfers of 0.01; reconciliation run.
Expected	Totals exact to the paisa; reconciliation OK.
Actual	Measured: one_paisa_transfers = **200** reconciliation_status = **OK**
Result	**PASSED** 10.25 s

P2-TC-3.1c	Fractional amounts in the JSON body
Test	`PrecisionBoundaryTest#fractionalAmounts_areRejectedNotTruncated`
Goal	A client sending rupees as a decimal (e.g. 33.33) must get an error, not a different amount.
How it runs	POST /transfers with amount 33.33, 0.5, 1e-2 and 3333.999 (amount is an integer number of paise).
Expected	400 for every fractional value; nothing moves.
Actual	FIXED (finding 5). Before the fix 33.33 and 3333.999 were accepted and truncated to 33 and 3333 paise. spring.jackson.deserialization.accept-float-as-int=false now rejects every fractional amount with 400. Measured: status_for_amount_33.33 = **400** status_for_amount_0.5 = **400** status_for_amount_1e-2 = **400** status_for_amount_3333.999 = **400** paise_moved_by_fractional_requests = **0**
Result	**PASSED** 0.78 s

P2-TC-3.1d	Amount boundaries
Test	`PrecisionBoundaryTest#amountBoundaries`
Goal	Edges of the accepted range behave exactly.
How it runs	0.01, the 100,000.00 maximum, maximum + 0.01, and a number beyond Long.MAX_VALUE.
Expected	201, 201, 400, 400 (no overflow, no 500).
Actual	As expected.
Result	**PASSED** 1.09 s

P2-TC-3.1e	No floating point in the money model
Test	`PrecisionBoundaryTest#noFloatingPointInMoneyModel`
Goal	Money must be held as scaled integers or BigDecimal, never float/double.
How it runs	Reflection over every compiled application class; information_schema check of all money columns.
Expected	Zero float/double fields; wallets.balance, transactions.amount, ledger_entries.amount/balance_after and idempotency_keys.balance_after are all BIGINT (paise). This satisfies the 'scaled integer' option in the plan, so BigDecimal is not needed.
Actual	Measured: money_columns_bigint = **5** float_or_double_fields = **0**
Result	**PASSED** 0.06 s

Appendix A. Full test inventory

Every test executed in this run, including the pre-existing regression tests.

Test	Case	Result	Time (s)
ConsumerReplayTest#notificationWorkerCrash_noDuplicateNotifications	P2-TC-1.1b	PASSED	7.57
ConsumerReplayTest#webhookWorkerCrash_merchantReceivesEventOnce	P2-TC-1.1a	PASSED	3.35
ContentionRecoveryTest#forcedDeadlock_isRetriedNot500	P2-TC-2.1b	PASSED	1.99
ContentionRecoveryTest#hotWalletWithInjectedDelay_allSucceed	P2-TC-2.1a	PASSED	17.45
DoubleSpendSuiteTest#concurrentDrain_exactlyAffordableTransfersSucceed	TC-1.1a	PASSED	1.60
DoubleSpendSuiteTest#idempotencyFlood_executesOnceAndReplaysSamePayload	TC-1.2	PASSED	1.67
DoubleSpendSuiteTest#opposingDirections_noDeadlockNoNegativeBalance	TC-1.1b	PASSED	5.47
DoubleSpendSuiteTest#ringOfThreeWallets_noDeadlock	TC-1.1c	PASSED	4.33
ErrorResponseTest#persistentConflict_is503	P2-TC-2.1e	PASSED	0.88
ErrorResponseTest#transientConflict_isRetried	P2-TC-2.1d	PASSED	0.97
ErrorResponseTest#unhandledError_is500	P2-TC-2.1c	PASSED	0.59
EventPipelineTest#publisherFailure_isSwallowed	TC-3.1d	PASSED	0.03
EventPipelineTest#publisher_sendsOnlyAfterCommit	TC-3.1c	PASSED	0.43
EventPipelineTest#redeliveredEvent_isIdempotent	TC-3.1b	PASSED	0.77
EventPipelineTest#workersDown_transfersSucceed_eventsProcessedAfterRestart	TC-3.1a	PASSED	3.91
IdempotencyEdgeCaseTest#clientTimeoutAfterCommit_retryReturnsOriginalResult	TC-2.1a	PASSED	2.25
IdempotencyEdgeCaseTest#failedRequestRollsBackKey_retryExecutes	TC-2.1c	PASSED	0.95
IdempotencyEdgeCaseTest#invalidKeyHeader_isRejected	TC-2.2e	PASSED	0.76
IdempotencyEdgeCaseTest#keysAreScopedPerUser	TC-2.2d	PASSED	1.49
IdempotencyEdgeCaseTest#normalisedEmail_isTreatedAsSameRequest	TC-2.2c	PASSED	0.73
IdempotencyEdgeCaseTest#retryWhileOriginalInFlight_getsSameResult	TC-2.1b	PASSED	1.96
IdempotencyEdgeCaseTest#sameKeyDifferentAmount_isRejected	TC-2.2a	PASSED	0.99
IdempotencyEdgeCaseTest#sameKeyDifferentRecipient_isRejected	TC-2.2b	PASSED	1.23
KafkaOutageTest#brokerDown_concurrentTransfersAllCommit	TC-3.1f	PASSED	17.25
KafkaOutageTest#brokerDown_transferStillSucceeds	TC-3.1e	PASSED	4.29
LedgerIntegrityTest#chaosThenReconciliation_reportsNoDiscrepancies	TC-4.1	PASSED	27.11
LedgerIntegrityTest#concurrentRun_isRefused	TC-4.3	PASSED	0.46
LedgerIntegrityTest#invalidAmounts_neverReachTheLedger	TC-4.4	PASSED	0.74
LedgerIntegrityTest#reconciliation_detectsInjectedCorruption	TC-4.2	PASSED	1.78
PrecisionBoundaryTest#amountBoundaries	P2-TC-3.1d	PASSED	1.09
PrecisionBoundaryTest#fractionalAmounts_areRejectedNotTruncated	P2-TC-3.1c	PASSED	0.78
PrecisionBoundaryTest#noFloatingPointInMoneyModel	P2-TC-3.1e	PASSED	0.06
PrecisionBoundaryTest#splitsAndMicroTransfersStayExact	P2-TC-3.1b	PASSED	10.25
PrecisionBoundaryTest#thirtyThreeThirtyThreeThreeTimes	P2-TC-3.1a	PASSED	1.78

Test	Case	Result	Time (s)
RateLimitTest#loginBurst_isThrottled	TC-5.2a	PASSED	0.94
RateLimitTest#readsAreNotThrottled	TC-5.2d	PASSED	0.59
RateLimitTest#redisDown_failsOpen	TC-5.2e	PASSED	0.63
RateLimitTest#registerBurst_isThrottled	TC-5.2c	PASSED	1.45
RateLimitTest#transferBurst_isThrottledPerUser	TC-5.2b	PASSED	1.99
ReconciliationTest#cleanLedger_reportsOk	TC-4.R1	PASSED	0.54
ReconciliationTest#corruptedBalance_isDetected	TC-4.R2	PASSED	0.43
SecurityBoundaryTest#algNone_isUnauthorized	TC-5.1e	PASSED	0.65
SecurityBoundaryTest#cannotDebitAnotherUsersWallet	TC-5.1h	PASSED	1.49
SecurityBoundaryTest#expiredToken_isUnauthorized	TC-5.1a	PASSED	0.58
SecurityBoundaryTest#foreignKey_isUnauthorized	TC-5.1d	PASSED	0.45
SecurityBoundaryTest#malformedOrMissing_isUnauthorized	TC-5.1f	PASSED	0.85
SecurityBoundaryTest#payloadSwappedToVictim_isUnauthorized	TC-5.1c	PASSED	0.57
SecurityBoundaryTest#roleEscalation_isUnauthorized	TC-5.1g	PASSED	0.48
SecurityBoundaryTest#tamperedSignature_isUnauthorized	TC-5.1b	PASSED	0.89
SecurityHttpTest#adminRole_canRunReconciliation	regression	PASSED	0.65
SecurityHttpTest#noToken_isUnauthorized	TC-5.R1	PASSED	0.03
SecurityHttpTest#refreshToken_worksOnlyOnce	TC-5.R3	PASSED	0.20
SecurityHttpTest#userRole_isForbiddenOnAdminEndpoints	TC-5.R2	PASSED	0.14
SecurityHttpTest#validToken_canReadProfile	regression	PASSED	0.29
TransferConcurrencyTest#concurrentTransfers_neverOverdraw	TC-1.R1	PASSED	0.95
TransferConcurrencyTest#insufficientFunds_isRejectedAndLeavesBalancesUntouched	TC-4.R3	PASSED	0.63
TransferConcurrencyTest#opposingTransfers_doNotDeadlock	TC-1.R2	PASSED	1.73
TransferConcurrencyTest#reusedKeyWithDifferentBody_isRejected	TC-2.R1	PASSED	0.53
TransferConcurrencyTest#sameIdempotencyKey_movesMoneyOnce	TC-1.R3	PASSED	0.81
TransferConcurrencyTest#transferToSelf_isRejected	TC-4.R4	PASSED	0.22
WalletApplicationTests#contextLoads	regression	PASSED	0.01
WebhookRetryTest#failsThreeTimesThenSucceeds_withExponentialBackoff	TC-3.2a	PASSED	14.44
WebhookRetryTest#permanentFailure_goesToDeadLetterTopic	TC-3.2c	PASSED	12.54
WebhookRetryTest#timeoutThenSuccess	TC-3.2b	PASSED	13.31

Appendix B. How to reproduce

1. Start the infrastructure: `docker compose up -d` (the tests use the `wallet_test` database).

2. Run the suite: `source env.sh && ./mvnw test`. The ledger soak defaults to 20 s; run the 1-hour version with `-Dchaos.seconds=3600 -Dtest=LedgerIntegrityTest`.

3. Rebuild this report: `python tools/test-report/generate_report.py`.

The application itself must not be running while the suite runs: it would join the same Kafka consumer groups that EventPipelineTest restarts.